MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE RUSSIAN FEDERATION
FEDERAL STATE AUTONOMOUS EDUCATIONAL INSTITUTION OF HIGHER EDUCATION
"NOVOSIBIRSK NATIONAL RESEARCH UNIVERSITY
STATE UNIVERSITY"
(NOVOSIBIRSK STATE UNIVERSITY, NSU)

09.03.02 - Informatics and Computer Engineering
Profile: Computer Science and System Design

TEAM PAPER

Authors:
Ilya Pereverzev
Bogdan Svirsky
Grigory Chernikov

Job topic:
“Snake Game”

Novosibirsk, 2024

STRUCTURE

Introduction **3**
Problem Statement **4**
Analogues **5**
Hardware **6**
Display controller 7
Blue part 9
Orange part 10
Pink part 11
Green part 12
Counter 13
Keyboard decoder 13
Computer 14
Software **16**
Snake's implementation 17
Snake's lifecycle 20
Hardware & Software interaction **25**
Software side 26
Hardware side 29
User's Manual **31**
Conclusion **32**
APPENDIX **33**

Introduction

Welcome to the world of "Snake Game", a classic game that rightfully gained popularity since its inception in 1977. The game, available for both single-player and two-player modes on arcade machines, offered a unique gameplay experience that impressed players around the world.

The rules of the game "Snake" were quite simple: control the snake and eat food. However, the simplicity of this concept hid an incredible addiction and challenge for players. Eating food led to the snake's growth, but the game field was limited, so to complete the game, it was necessary to fill the entire field with the snake.

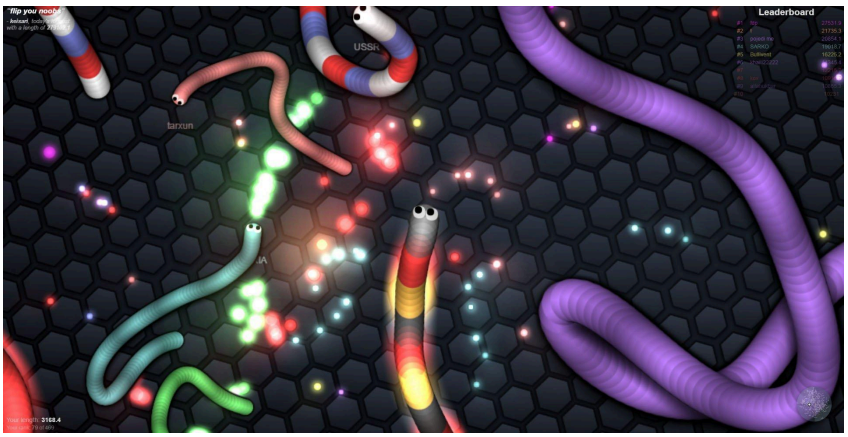
Problem Statement

The snake game interested us the most. We want to implement a classic snake with user-friendly controls using four buttons WASD. We have not seen any snake implementations on a CdM16 processor and we considered this to be a problem that needs to be solved, so we decided to develop a game in Logisim on a 16-bit processor, with it we can increase the speed of code execution through the emulator, we also have more registers and a lot of memory. Also this gives us the opportunity to develop our project in the future.

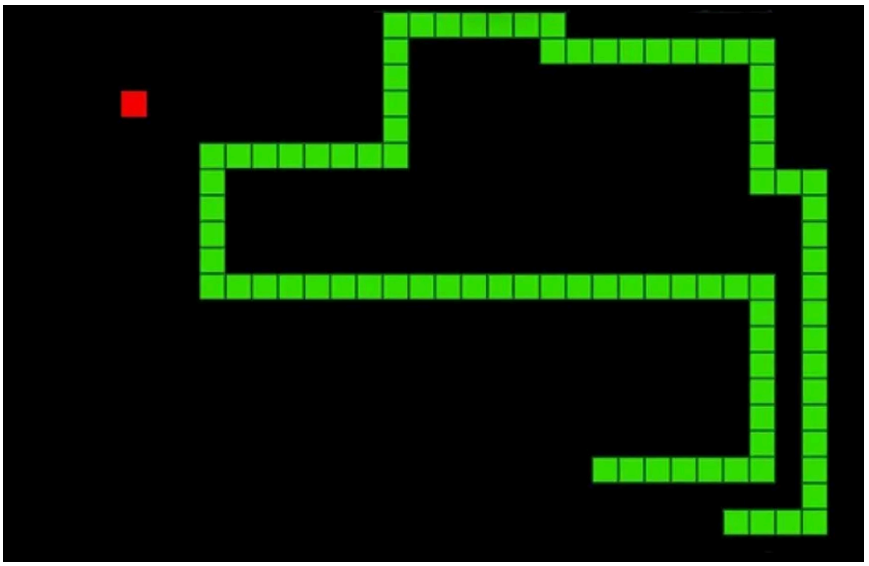
Analogues

Snake surely has a lot of implementations on different platforms, different languages, different architectures. Rules also can vary from one snake to another.

1. Slither.io is a snake game with multiple players.



2. Original snake game. This is how a snake looked the first time it was created.



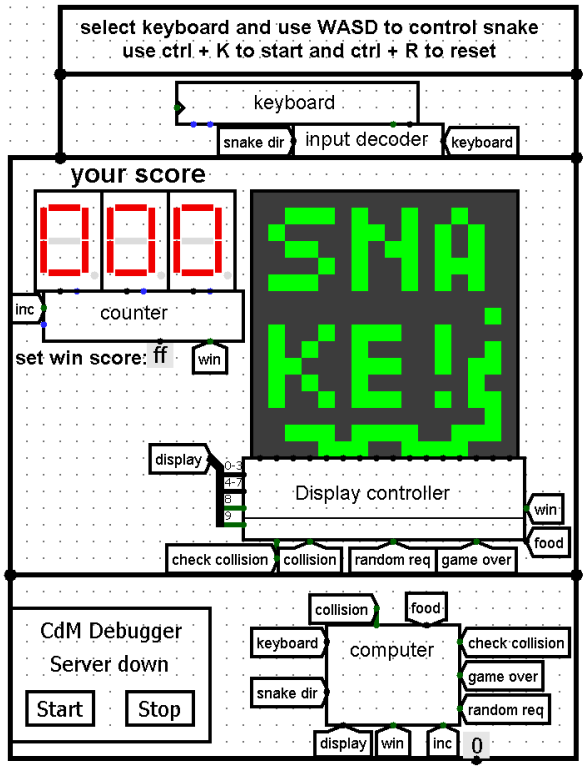
We think our game is the first implementation of the snake on the CdM16 processor. There are few things more are doing our project different:

1. We have 4 keyboard buttons to control the snake (in the original snake there were 2 buttons, the snake moves forward on its own or, if the player presses left or right, the snake will move left or right).
2. Our snake game has the possibility to set a custom win score, so players can choose difficulty.
3. Due to the fact that we use this processor, we can develop our game further
4. We have a good random food placing system, so players can score the number of points up to the maximum (255) and no worry about bugs.

Hardware

In our project we have 9 circuits:

- main
- memory_map_extrude
- random_generator
- keyboard_decoder
- computer
- display_controller
- reverser
- decoder
- counter



In the main scheme we have implemented a game area in which the user can play our snake game.

To start our game, you just need to run the scheme with the keyboard shortcut ctrl + K.
To control the snake use WASD.
If you lose or win, you can restart the game using the keyboard shortcut ctrl + R.

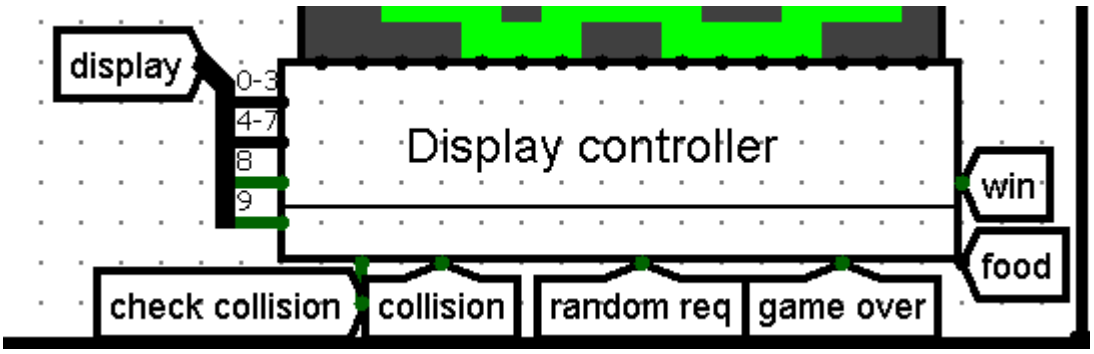
Here we have a display, keyboard, scoreboard.
The display is 16 by 16.
Our display is connected to the display controller and decoder.

The keyboard is connected to an input decoder.

Also we have the computer, this is the brain of our scheme.

Display controller

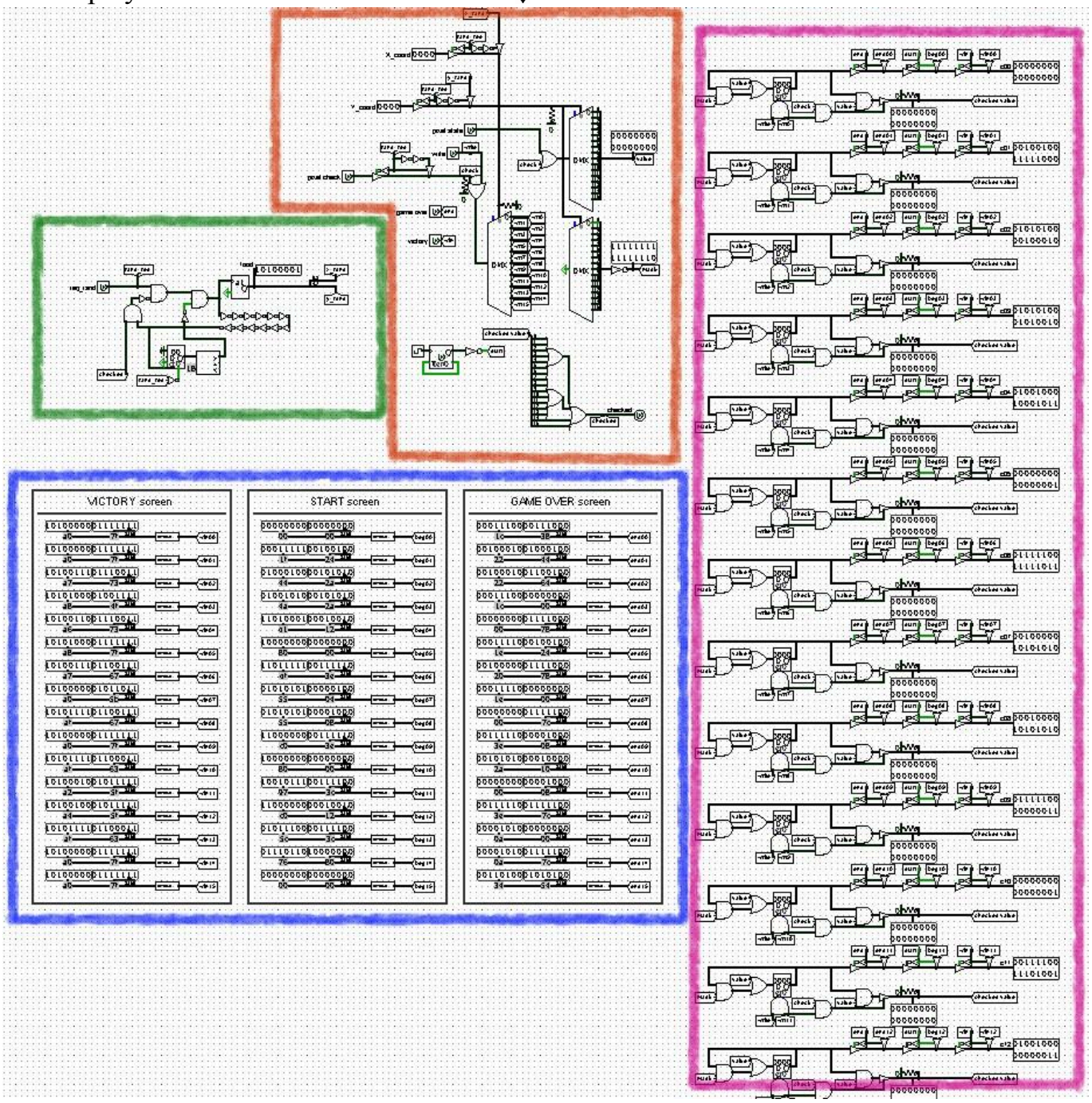
Let's take a closer look at the display controller.
We use this subcircuit for all operations that we need to do with display state. We can draw and erase pixels, get the information about certain pixels, get random coordinates of new food, switch current display to defeat or victory screen.



It takes on several inputs:

- signals from the decoder - two 4-bit values: x and y coordinates of pixel, 1-bit pixel state signal and 1-bit write signal.
- check collision - signal that is used by the processor to get the information about the pixel that was selected by x and y coordinates: is it enabled or disabled.
- collision - result of the ‘check collision’ operation.
- random req - this input is used by the processor to request a random byte with food coordinates.
- game over - defeat signal, enables defeat screen
- win - victory signal, enables victory screen
- food - food coordinates, result of the ‘random req’ query

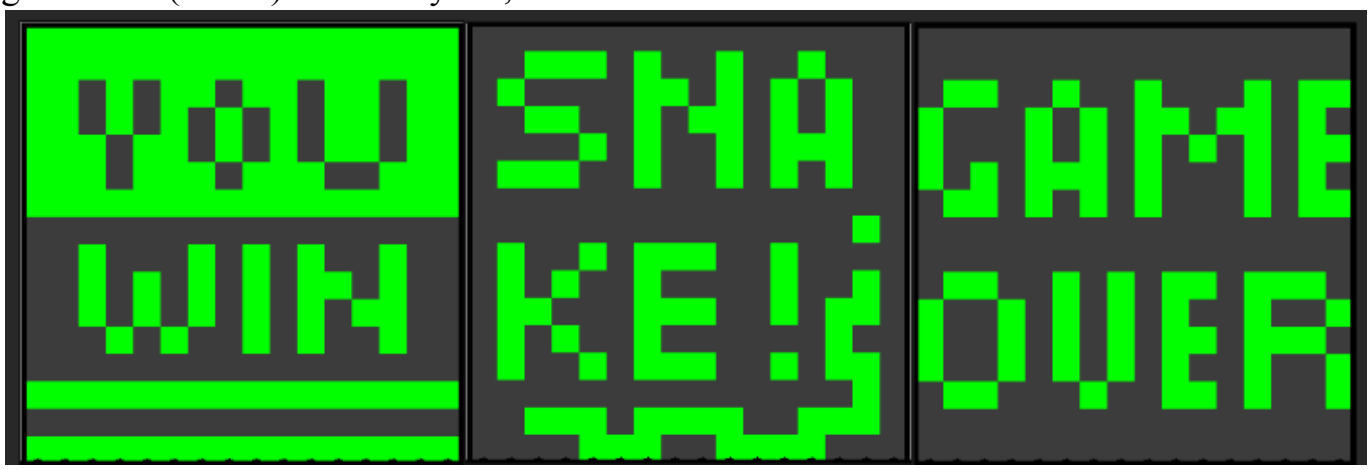
The display controller looks like this inside↓

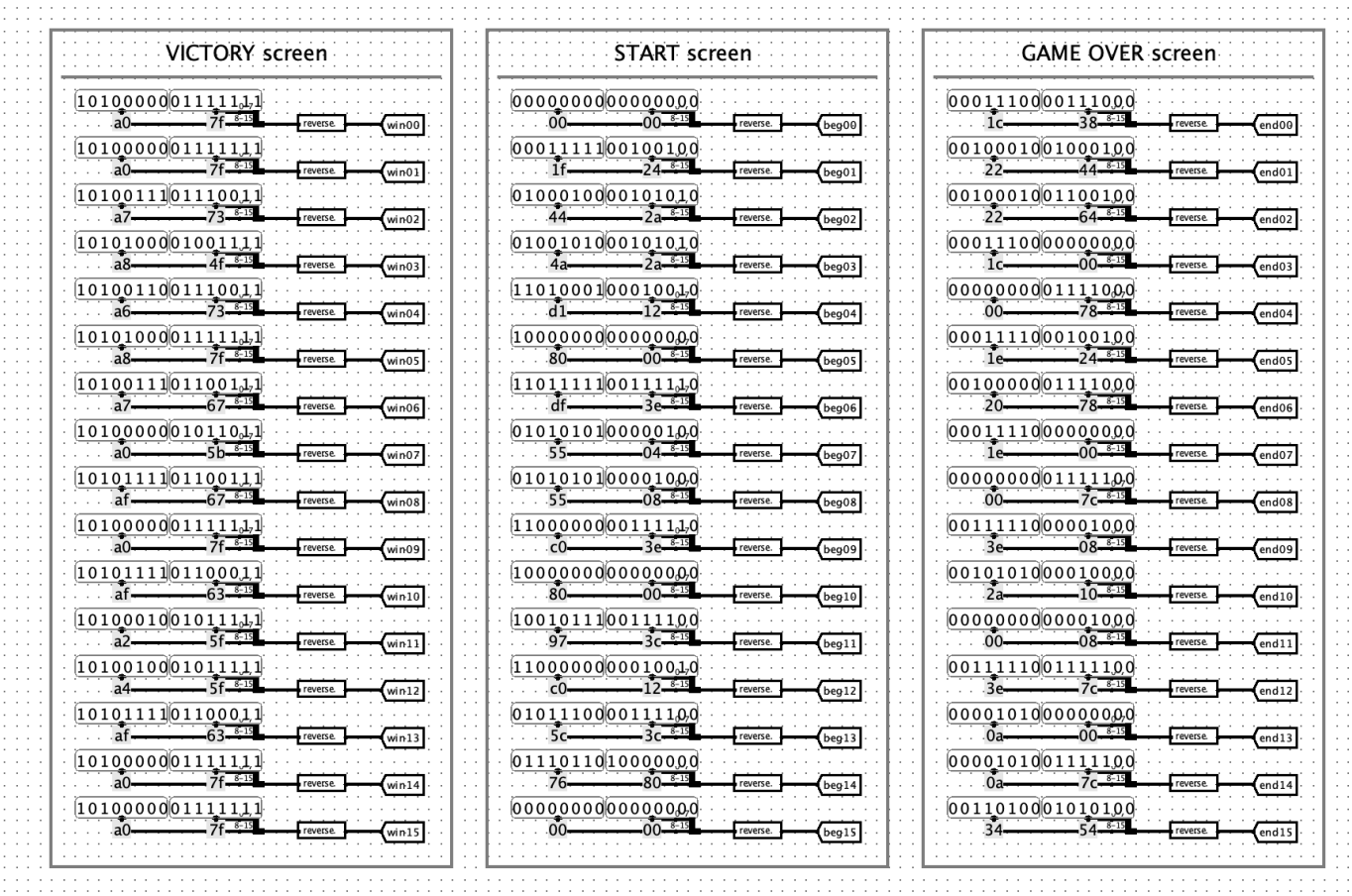


It splits to the several semi-independent parts, by their semantics. Let's take a closer look at each part.

Blue part

three displays (constants that store all the information needed). Victory, start and game over (defeat). Here they are, inside and outside.



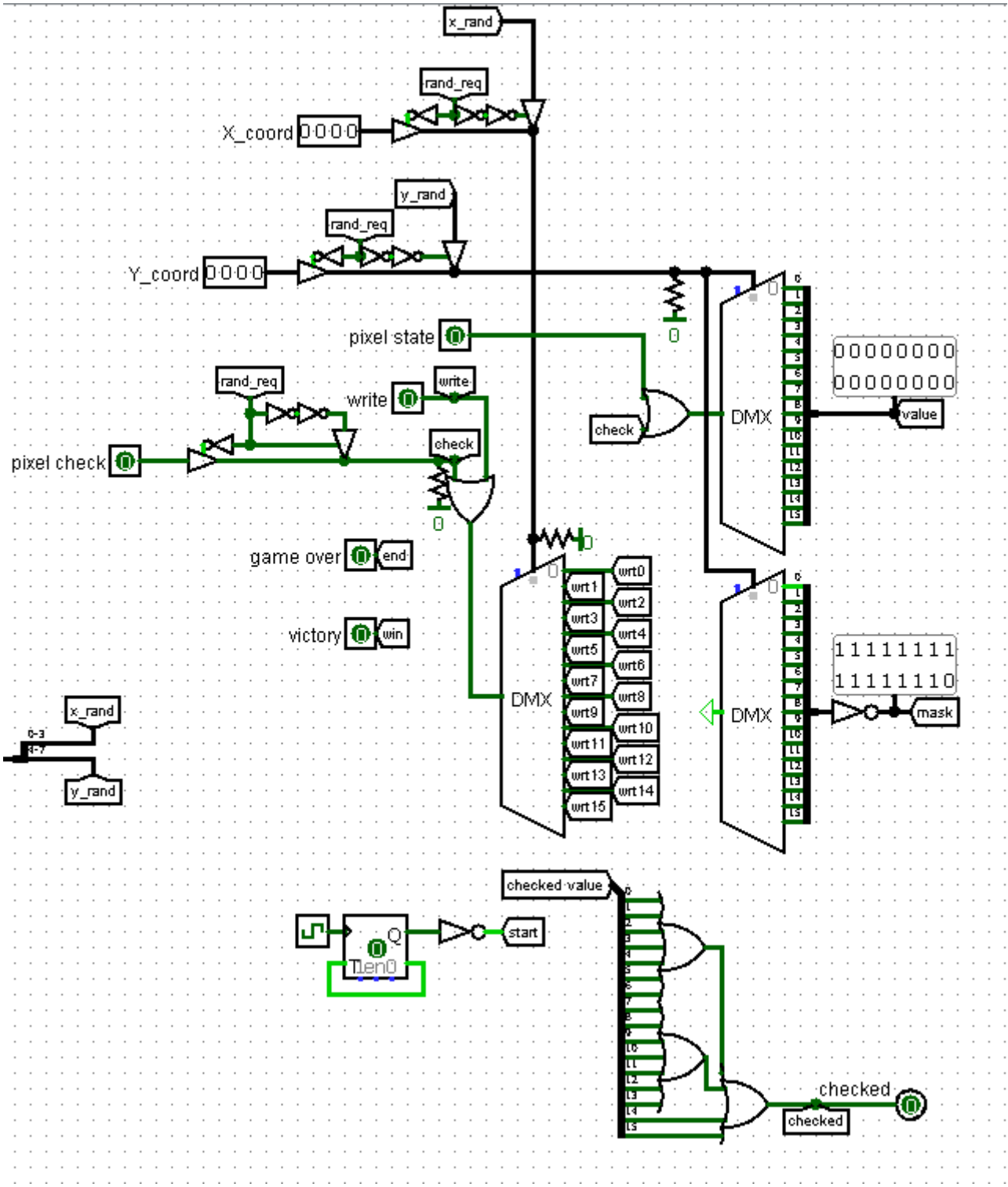


Orange part

main controller logic, that implements two main things:

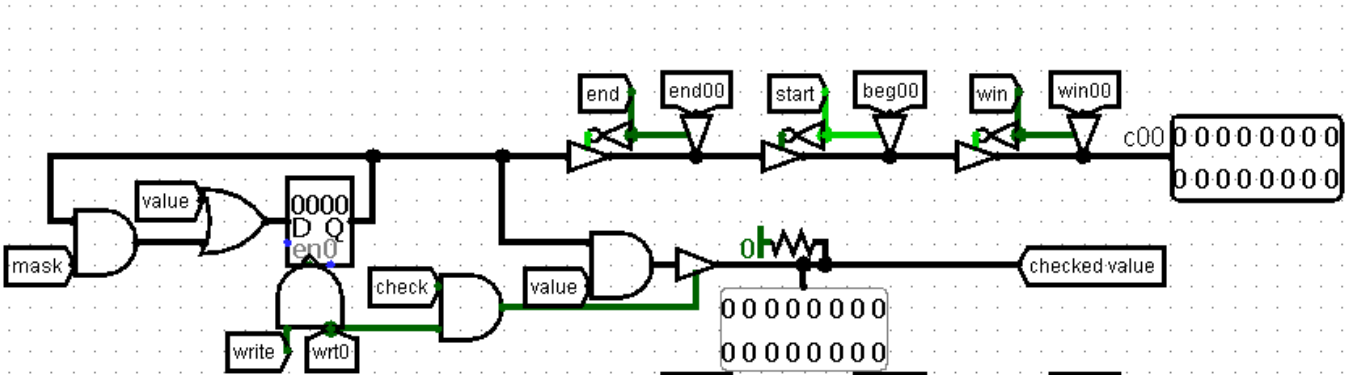
- drawing new pixels on display (or erase pixels that were drawn)
- checking pixel with given coordinates (is it enable or disable)

Here it is.



Pink part

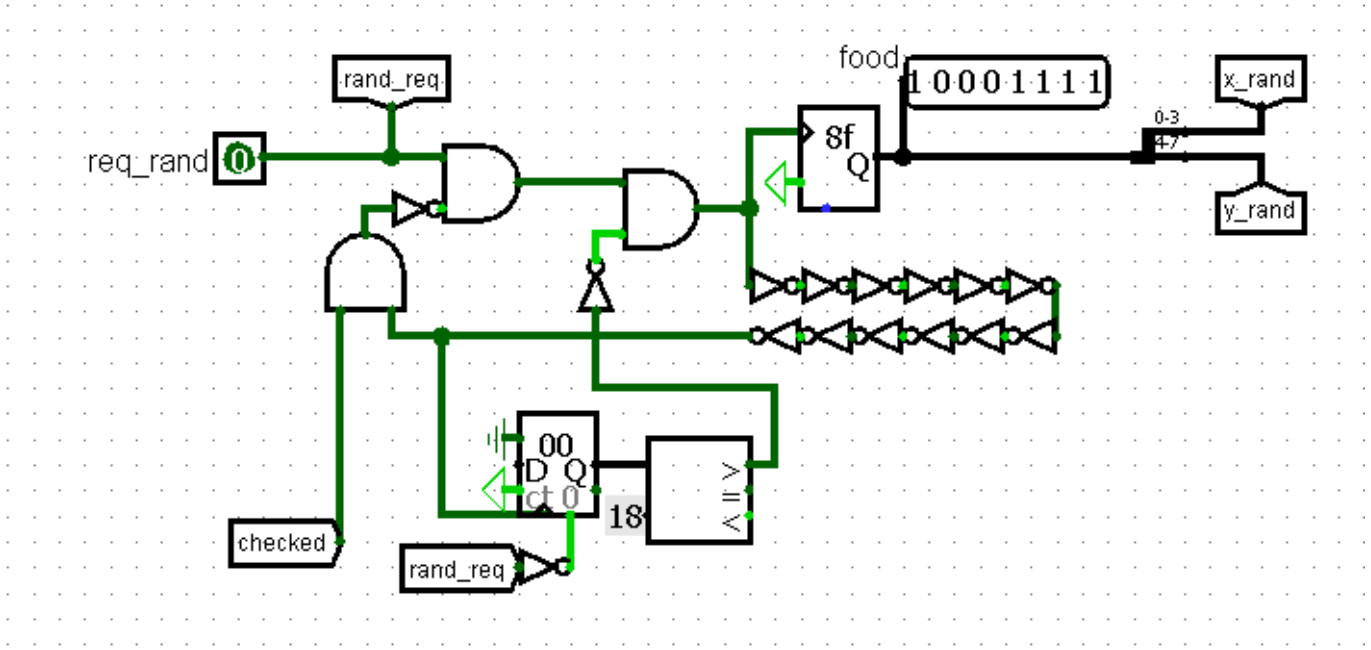
It is the biggest, but the simplest part of this subcircuit. It fully stores the current display state and provides possibilities to modify it and to check any pixel state separately. It consist of repeating elements like this



One element controls one column of the display. For example, this one controls column number 0. Output ‘c00’ goes directly to the input of the appropriate column of the display. It takes a lot of data from the other parts, mostly the orange part as controlling one. You can see ‘end’, ‘start’ and ‘win’ gates that receive the data from the blue part (with displays) and are controlled by the orange part. Lower part of this element is used for checking the value of the selected bit.

Green part

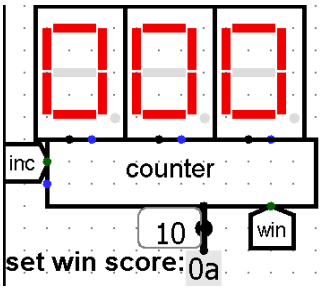
Smallest part of the circuit that implements the most interesting things.



This part of the subcircuit is responsible for the generation of the random coordinates for food. The problem is: not all random coordinates could be used as food coordinates. For example, we cannot put food on the snake itself. That's why after every generation we should ask the orange part - is this pixel already occupied by the snake or not. For this purpose we use pixel check functionality. If the pixel is not suitable for food placing, we can simply generate new coordinates and check them by the same way.
But if we do all this pretty complicated stuff by the processor, it will take a lot of execution time. As we have tested, if the most part of the display is covered by enabled pixels, the processor will find a place for food in several seconds - from 1 to 10. It is too long, players don't want to wait, players want to play!
That's why we came up with this idea. We can increase the speed of the search for the suitable pixel a lot, if we will not use a processor and even a clock for this purpose - only naked logic. We will generate new random coordinates until we find a suitable pixel or 24 attempts (we cannot do more, Logisim counts it as "oscillation apparent" by default settings). All those generations will happen in the time of one tick - as fast as possible. If it can't find a suitable pixel in 24 attempts, the processor will ask the display controller to generate new food coordinates. It will slow the circuit, but not to the 10 seconds, much less.

Counter

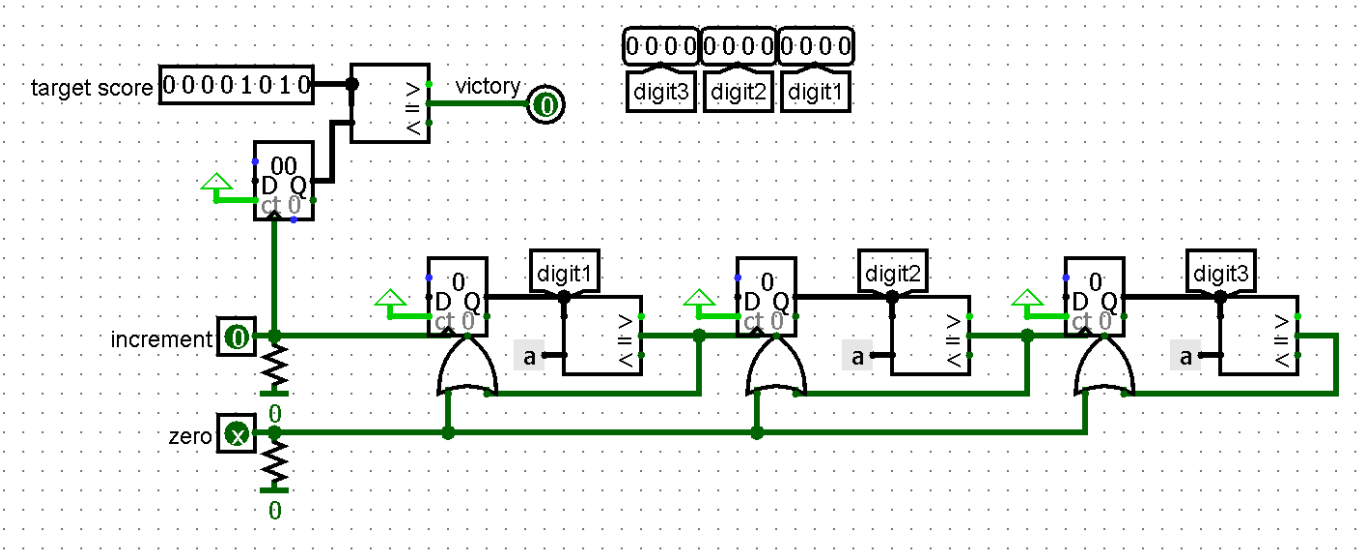
Let's take a look at the next subcircuit - counter.



We use it to show the player's score and to get a victory signal. There are several pins:

- Increment - when snake eat the food, we increment the counter.
- Target score - player can set this value as the level of complexity (constant below the counter, player can input any decimal value in constant.
- Victory - when a player achieves a target score, the counter raises this pin and the victory screen appears.
- 3 pins to control three 7-segment displays.
- The counter also has a 'zero' pin that can be used to clear the current value and set zero.

Counter looks like this inside

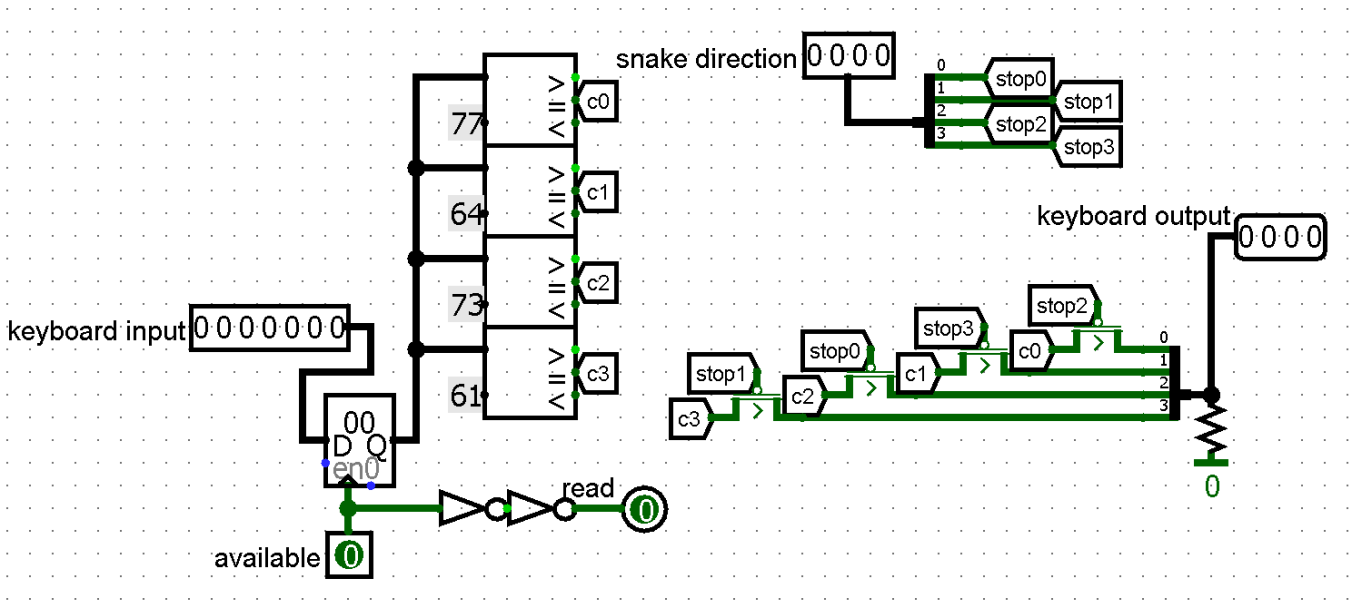


When the first digit becomes equal to 0xA (10), we clear it and transfer the '1' to the next digit by incrementing it. Same with the second digit.

Keyboard decoder

To control the snake properly, we need to store somewhere current direction of the snake movement. Also the snake cannot turn backwards, all attempts to do this should be ignored. To solve these problems we came up with the following subcircuit - keyboard

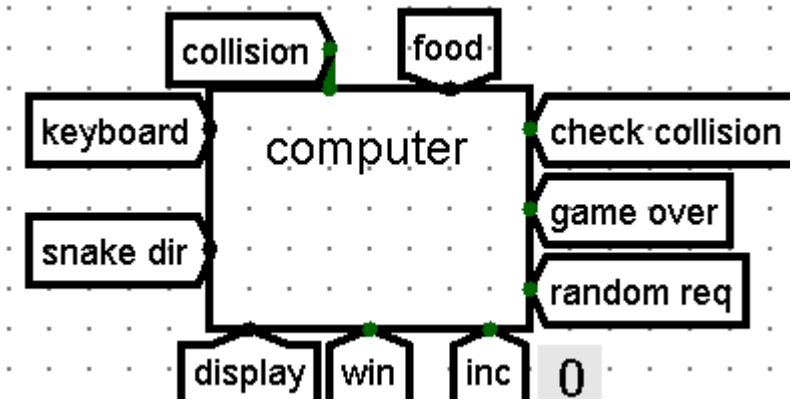
decoder.



We scan the pressure of one of the WASD buttons. Then decode it and if it isn't attempt to turn snake backwards on 180 degrees, we change value 'keyboard output'
In the 'snake direction' pin input goes current direction, that processor stores somewhere in memory. We need this value to detect when there is an attempt to turn backwards. As soon as there is something in keyboard, we scan it to the register.

Computer

There is kinda control module in our circuit, called 'computer'



We won't go deep inside it now, we'll do it in the chapter after next one. Inside of it there is a CdM16 processor connected to the memory in a Harvard manner. We use a processor to control the snake movement, length and detect when the snake eats the food.
Let us talk about inputs and outputs.
Inputs:

- win - 1-bit value that tells the processor about the victory. Processor halts after this value was taken from 0 to 1.

- keyboard - 4-bit value that indicates the current direction of the snake.
- collision - 1-bit value, result of the collision check received from the display controller.
- food - 8-bit value, coordinates of the new food received from the display controller after the request.

Outputs:

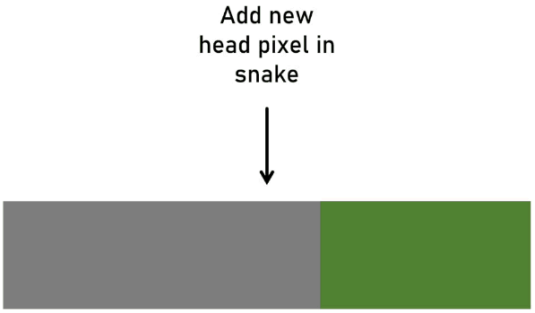
- display - 16-bit value that we use to erase and draw pixels. It contains x and y coordinates, pixel state bit and write signal bit.
- snake direction - 16-bit value from the r4 register that contains the current direction of the snake.
- random req - 1-bit value that tells the display controller to find free space and generate the new food coordinates there.
- check collision - 1-bit value that we use to check is some certain pixel enabled or disabled. We use it to detect snake collisions with itself or food.
- inc score - 1-bit output that is used for incrementing the score.
- game over - 1-bit value by what the processor signals that there was a collision of the snake with itself or wall.

Software

In our project we use CdM16 processor for implementing the snake logic (movement, eating food and snake's collisions with walls and itself). Before we begin to analyze the implementation of game logic, it's necessary to understand interaction between the processor and the circuit.

Snake’s implementation

At the beginning of the development, we faced the problem - how to present a snake in code? At first, we wanted to realize the module in circuit, which will store snake’s pixels coordinates. But it's a bad idea, because we would also need to implement something like RAM, requests to it, etc. We decided not to implement any additional modules for snake in the circuit. Next idea was to use a simple byte array to store pixel coordinates, but we couldn't figure out how to optimally store data and work with it quickly. The final idea was to use the queue data structure to store pixels coordinates, because if we analyze the snake's movement on the screen, we will notice that the lifecycle of pixels included in a snake reminds the lifecycle of queue elements: we add a pixel to the beginning of the snake, it goes through the entire snake and comes out at its end - like elements in queue.



But how to implement a queue in code?

At first we wanted to use the processor’s stack. Pushing an element to a queue - instruction *push <register>*. For popping elements from the queue we need to store the count of popped elements to calculate the address of the first element in the queue. But in this implementation we can’t use stack for other tasks (stack must contain only the pixels coordinates), which can be useful when part of the registers is constantly occupied by external devices.

Because of this, the queue is implemented as a circular buffer. This is the most suitable implementation of queue for our needs. We know of its maximum size and are able to allocate the required amount of bytes.

```
snake_coords:
    ds 256
start_snake_coords: # pointer to first coords
    dc snake_coords
end_snake_coords: # place for new coords
    dc snake_coords
snake_length:
    dc 0
snake_direction:
    dc 0x10
```

- *snake coords* — the byte array on which the circular buffer runs
- *start snake coords* — address of the first element of queue (last pixel in snake)
- *end snake coords* — address of the last element of queue (head pixel of snake)

During initialization, the queue is empty and these addresses are equal and point to the first byte of the array.

```
update_coords_pointers> # store pointer in r2
    ldi r3, snake_coords
    sub r2, r3
    ldi r5, 255
    if
        cmp r3, r5
    is eq
        ldi r2, snake_coords
    else
        inc r2
    fi
    rts
```

The main part of the circular buffer - pointer's management. When one of the addresses (*start* or *end snake coords*) reaches the end of the byte array, we need to move it to the beginning. We declare that the subroutine works with the value in *r2* as with an address and the result of its work will be also stored in r2.

```
push_coords>
    ldi r2, end_snake_coords
    ld r2, r2
    stb r2, r0
    jsr update_coords_pointers
    ldi r3, end_snake_coords
    st r3, r2
    jsr turn_on_pixel
    jsr push_coords_to_display
    rts
```

Above, you can find the subroutine for adding a pixel to the snake. First, we store the new head pixel coordinates at the beginning of the queue. Next, we update the address of the end of the queue and store it in *end_snake_coords*. Finally, we need to display the new pixel - this is done by calling a separate subroutine.

```
pop_coords>
    push r0
    ldi r2, start_snake_coords
    ld r2, r2
    ldb r2, r0
    jsr turn_off_pixel
    jsr push_coords_to_display

    ldi r2, start_snake_coords
    ld r2, r2
    jsr update_coords_pointers
    ldi r3, start_snake_coords
    st r3, r2
    pop r0
    rts
```

Also we need to pop pixels from the snake. To do this, we implemented a separate subroutine in our code. At the beginning, we save value from *r0* on the processor's stack, as it always represents the current coordinates of the snake's head. After that, we get the last pixel in the snake and turn it off on display. At the end, we need to update the address of the end of the queue, store it in *start_snake_coords* and return *r0* to its previous value.

Snake's lifecycle

Now we can implement game logic. Program begins with snake's initialization:

```
init_snake>
    ldi r0, 0
    jsr push_coords
    ldi r1, 1
    ldi r2, snake_length
    st r2, r1
    clr r4
    rts
```

Snake starts her life from pixel *(0, 0)*. We push these coordinates in a queue and set the snake's length to 1. Also clear register *r4*, because at the beginning the snake hasn't direction.

After initialization we run subroutine for generating new food:

```
generate_food>
    push r0
    do
        ldi r6, 1
        ldi r6, 0
        ldi r1, external_devices_data + 1
        ldb r1, r0
        jsr check_pixel
    until z
    jsr turn_on_pixel
    jsr push_coords_to_display
    pop r0
    rts
```

Here we also need to store value from register *r0* to processor's stack, because it's current head coordinates. After this, we generate new 8-bit length value (switch bit 0 in r6 from 1 to 0) and check - is it value collision with snake:

```
check_pixel>
    ldi r6, 0b10
    ldi r1, external_devices_data + 2
    ldb r1, r1
    ldi r6, 0
    tst r1
    rts
```

We set bit 1 in register *r6* to 1 and load byte, which is connected to the display controller and stores the result of checking. Result of checking - *tst r1*.

After this, we generate a new value if the pixel is already on (because it means that it applies to snake or old generated food) or turn on the pixel of new generated food.

```
main>
    jsr set_external_devices
    jsr init_snake
    jsr generate_food
    while
        ldi r1, snake_length
        ld r1, r1
        ldi r2, 256
        cmp r2, r1
    stays gt
        jsr move_snake
    wend
end.
```

Okay, we initialized the snake and generated the first food. Now we need to start the game process. The snake "lives" in the program until it fills the entire screen (because of this, in a while loop we compare its length with 256 (16x16 display). In each iteration we move snake:

```
move_snake>
    ldi r1, external_devices_data
    ldb r1, r1
    if
        tst r1
    is z
        if
            tst r4
        is z
            rts
        else
            move r4, r1
        fi
    fi

    move r1, r4
```

At the beginning of the movement, we load data from the keyboard (first byte of memory mapped I/O). After this, we process this data: if it's equal to zero and snake direction is equal to zero, we will stop the subroutine, because we caught the situation when the snake hasn't yet started to move. If from the keyboard we get zero, but the snake has direction, we move the value from *r4* to *r1*, because we need to move the snake in the

previous direction. After processing *r1* and *r4*, we store the value, in which we will be moving, in *r4* (it becomes the current snake direction). Let's move on the next part of subroutine:

```
ldi r2, 0b1
if
    cmp r1, r2
is eq
    jsr move_up
else
    ldi r2, 0b10
    if
        cmp r1, r2
    is eq
        jsr move_right
    else
        ldi r2, 0b100
        if
            cmp r1, r2
        is eq
            jsr move_bottom
        else
            jsr move_left
        fi
    fi
fi
```

Here we determine which direction to move the snake. For each direction we have subroutine:

```
move_up>
    ldi r1, 0b11110000
    and r0, r1
    ldi r2, 0b11110000
    if
        cmp r2, r1
    is eq
        jsr game_over
    else
        ldi r1, 0x10
        add r1, r0
    fi
    rts
```

```
move_bottom>
    ldi r1, 0b11110000
    and r0, r1
    if
        tst r1
    is eq
        jsr game_over
    else
        ldi r1, 0x10
        sub r0, r1
        move r1, r0
    fi
    rts
```

```
move_left>
    ldi r1, 0b1111
    and r0, r1
    if
        tst r1
    is eq
        jsr game_over
    else
        ldi r1, 0b1
        sub r0, r1
        move r1, r0
    fi
    rts
```

```
move_right>
    ldi r1, 0b1111
    and r0, r1
    ldi r2, 0b1111
    if
        cmp r2, r1
    is eq
        jsr game_over
    else
        ldi r1, 1
        add r1, r0
    fi
    rts
```

In all of these subroutines you can see coordinates check. When snake's head stands next to the wall and will go beyond the display on next step we run the game over's subroutine:

```
game_over>
    ldi r5, 0b100
    or r5, r6
    halt
    rts
```

This code simply switches bit 2 in register *r6* and halts the processor.
Okay, we have moved snake's head and can goes to the end of snake's movement:

```
ldi r1, external_devices_data + 1
ldb r1, r1
if
    cmp r1, r0
is eq
    jsr push_coords
    jsr eat_food
else
    jsr check_pixel
    if
        tst r1
    is z
        jsr push_coords
        jsr pop_coords
    else
        jsr game_over
    fi
fi
rts
```

```
game_over>
    ldi r5, 0b100
    or r5, r6
    halt
    rts
```

In this part of the *move_snake* subroutine, we check for a collision between the coordinates of the snake's head and those of the food. If they match we add this new pixel in snake (*push_coords*, pixel is already turned on, but we need to store its coordinates in queue) and "eat food":

```
eat_food>
    ldi r1, snake_length
    ld r1, r1
    inc r1
    ldi r2, snake_length
    st r2, r1
    ldi r6, 0b10000
    clr r6
    ldi r1, external_devices_data + 3
    ldb r1, r1
    tst r1
    bnz win
    jsr generate_food
    rts
```

```
win>
    halt
    rts
```

Firstly, we increment the snake's length. After this, we increment the user's score by signal to counter (switching bit 4 from 1 to 0). Finally, we need to check: has the user won or is he still playing? If the win flag is 1 (value in *s03*, from memory mapped input), we stop the processor (*win* subroutine). Otherwise, we generate new food and continue working.

If the new snake’s coordinates aren’t equal to the food's coordinates, we check: did the snake crash into itself (*jsr check_pixel*)? If it is, we set the game over screen and stop the processor (*jsr game_over*). Otherwise, we push new coordinates to the snake (*jsr push_coords*) and pop the oldest pixel from snake (*jsr pop_coords*).

Hardware & Software interaction

Program executes on the cdm16 processor. Processor interacts with the circuit by the input and output.

For the input we use memory mapping technique. At the beginning of execution the processor sets the starting address of memory space by the process of initialization that was described in the Hardware Side (chapter after the next). Programmer can consider this space as a read only array of 16 bytes length and use ‘ldb’ instruction to read one byte from memory mapped space.

For the output we use three registers of the processor. One (r5) we use only one time at the beginning to set the starting address of memory mapped space and three (r0, r4 and r6) are used constantly, r0 to control the display, r6 to control counter and check snake collisions, r4 is used to control back movement of the snake.

Software side

The logic of processor-circuit relationship includes the following agreements:

1. Processor's register *r0* is used for display controller management:

   The first *four bits* are the X coordinate of the pixel, next *four bits* - Y coordinate (*r0* should always be like this).

   Eighth bit for pixel mode - 0 for off, 1 for on.

   Ninth bit for transfer pixel to display controller - switch it from 1 to 0

```
push_coords_to_display>
    ldi r2, 0x200
    or r2, r0
    sub r0, r2
    move r2, r0
    ldi r1, 0xff
    and r1, r0
    rts
```

   Above you can see a subroutine for transferring current pixels in *r0* to display. It switches bit 9 from 1 to 0 and puts r0 to a standard form by two last instructions.

   Also, we wrote auxiliary subroutines to manage pixel states. They perform simple actions - either setting *bit 8* of *r0* to 1 or 0:

```
turn_on_pixel>
    ldi r2, 0x100
    or r2, r0
    rts
```

```
turn_off_pixel>
    ldi r2, 0xff
    and r2, r0
    rts
```

2. Processor's register *r6* is used for external devices management:

   Zero bit is used for signal (switch 1 to 0) to generating new value from random generator

First bit is used for checking collision for current coordinates in *r0* (set to 1)
Second bit is used for setting game over screen - set to 1
Third bit is used for setting address of external device's data (next point) - switch it from 1 to 0

3. Processor's register *r5* is used once for setting address for memory mapped input/output:

```
external_devices_data:
    ds 16
```

At the beginning of the program, we allocate 16 bytes (because our memory mapped input/output is 16 bytes) for data from external devices.
We declare that these bytes don't store processor information (because they are allocated in code) and that we will only read data from them.

```
set_external_devices>
    ldi r5, external_devices_data
    ldi r6, 0
    ldi r6, 0b1000
    ldi r6, 0
    rts
```

After that, we need to transfer the address of allocated data to a memory mapped module — switch *third bit* on *r6* from 1 to 0
Now we can use simple instructions to get data from external devices like that:

```
ldi r1, external_devices_data + 1
ldb r1, r1
```

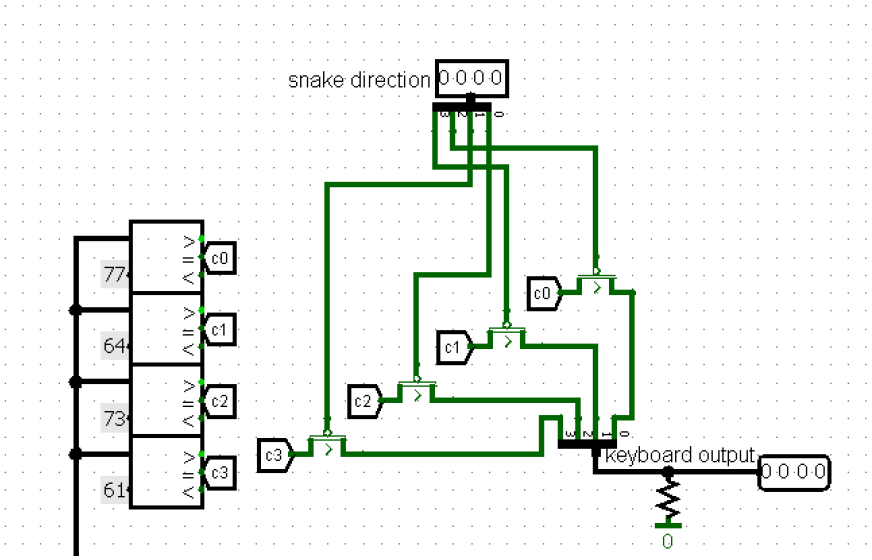
4. The address under the label *external_devices_data* is used for getting data from external devices:

Zero byte (address equals to *external_devices_data*) - keyboard value from keyboard decoder

First byte - 8-bit length random value, which generated from random generator

Second byte - checking collision from display controller (1 or 0)

5. Also we use register *r4* for transfer current snake direction to keyboard decoder for disabling back movement for snake

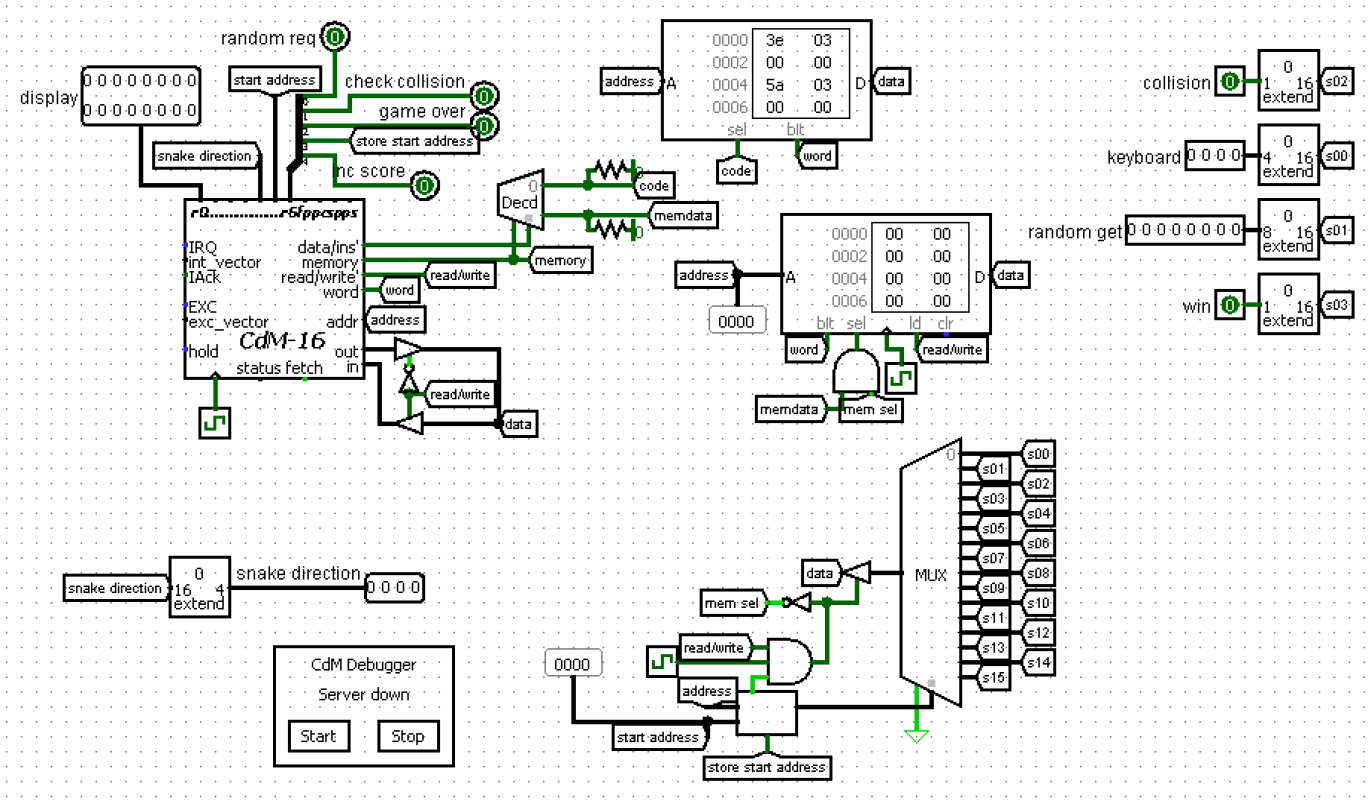


Above you can see part of the keyboard decoder. If the current direction of the snake is opposite to the entered value, then we set *0b0000* on the keyboard output and process this case in program. How can we verify that they are opposite? Simple matching:

- *0b1000* blocks *0b0010*
- *0b0100* blocks *0b0001*
- *0b0010* blocks *0b1000*
- *0b0001* blocks *0b0100*

Hardware side

Let's take a closer look at the 'computer' subcircuit from the inside. We chose to have Harvard architecture to make game restart easier. In harvard architecture



we use ROM, data in ROM doesn't erase when the user presses ctrl + r and restarts the circuit, so we will start the game from the starting screen every time when ctrl + r was pressed.

For the output we chose to use 3 registers: r0, r4, r6 and r5 (last one only at the beginning).

- r0 is used to control the display - all its values go to the decoder and then to the display controller.
- r4 is used to control back movement of snake - its value goes to keyboard decoder
- r5 is used only once, at the beginning of execution to set the starting address of memory mapped address space.
- r6 is used to control different parts of the circuit such as score, collision checking, game over screen, start address storing.

For the purpose of sending data into the processor we chose to use memory mapped input with some improvements. We made it possible to set any address in memory as the starting address for memory mapped space. We come up with the following convention: at the beginning of program execution the processor should set the starting address. Consider A as the beginning of memory mapped address space. This space is 16 bytes length in memory (on processor's view). To set A, first r5 becomes equal to A, that will be set, and then bit 3 on r6 should be taken from 0 to 1 (we use r6 as register for output).

Then the processor has 16 memory cells starting with A, like an array in memory, reading any data from that array will read data from tunnels s00 - s15 (you can look at them in the right down corner of the picture).

User's Manual

1. Install Logisim
2. Clone our repository from GitHub
   (*git clone https://github.com/BogdanSvirsky/snake-game.git*)
3. Open Logisim and select our main.circ file
4. Turn on the simulation (recommended 4.1Khz)
   (top bar -> Simulate -> Tick Frequency -> 4.1KHz)
5. To start playing just click on the keyboard and press any of WASD buttons
6. To reset the game you should press CTRL + R
7. Enjoy!!!

Conclusion

We have implemented the snake game in Logisim based on the CdM-16 processor.
We developed the logic of the snake game:

- Made a process of food eating
- Made a snake growth
- Made random food generation
- Made a scoreboard
- We have collision with snake's body and walls

As a result, we have 324 lines of code and 9 circuits.

APPENDIX

```
asect 0
main: ext                # Declare labels
default_handler: ext     # as external

# Interrupt vector table (IVT)
# Place a vector to program start and
# map all internal exceptions to default_handler
dc main, 0               # Startup/Reset vector
dc default_handler, 0    # Unaligned SP
dc default_handler, 0    # Unaligned PC
dc default_handler, 0    # Invalid instruction
dc default_handler, 0    # Double fault
align 0x80               # Reserve space for the rest
                         # of IVT

# Exception handlers section
rsect exc_handlers

# This handler halts processor
default_handler>
    halt

# Main program section
rsect main
# r0 - display managment (by decoder)
# r6 - external devices managment
# snake's coordinates - 0x00YX
snake_coords:
    ds 256 # size of display - 16 x 16, snake can fill it all
start_snake_coords: # address of first coords
    dc snake_coords
end_snake_coords: # address of place for new coords
    dc snake_coords
snake_length:
    dc 0
snake_direction: # current direction of snake movement
    dc 0

external_devices_data: # allocating bytes for memory mapped I/O
    ds 16
```

```
update_coords_pointers> # updating start or end address of coords in
circular buffer, need to store address in r2 (result also here)
    ldi r3, snake_coords
    sub r2, r3
    ldi r5, 255 # size of display - 1
    if
        cmp r3, r5
    is eq
        ldi r2, snake_coords # addres moves to beginning
    else
        inc r2 # address goes on
    fi
    rts

turn_on_pixel> # change pixel mode in r0
    ldi r2, 0x100
    or r2, r0 # up bit 8 to 1 <=> set pixel's mode to on
    rts

turn_off_pixel> # change pixel mode in r0
    ldi r2, 0xff
    and r2, r0  # down bit 8 to 0 <=> set pixel's mode to off
    rts

push_coords_to_display> # transfer pixel in r0 to display, r0 - 0bMYYYYXXXX
(M - mode)
    ldi r2, 0x200
    or r2, r0
    sub r0, r2 # signal to display controller to transmit the pixel
    move r2, r0
    ldi r1, 0xff
    and r1, r0 # put r0 to the standart state (0x00YX)
    rts

push_coords> # push new pixel to snake
    ldi r2, end_snake_coords
    ld r2, r2
    stb r2, r0 # push new pixel in snake
    jsr update_coords_pointers
    ldi r3, end_snake_coords
```

```
    st r3, r2 # update address end_snake_coords in cyclic buffer
    jsr turn_on_pixel
    jsr push_coords_to_display # turn on new pixel on display
    rts

pop_coords> # pop last pixel from snake
    push r0 # save value from r0, because it's current head coords
    ldi r2, start_snake_coords
    ld r2, r2
    ldb r2, r0 # get last pixel from snake
    jsr turn_off_pixel
    jsr push_coords_to_display

    ldi r2, start_snake_coords
    ld r2, r2
    jsr update_coords_pointers
    ldi r3, start_snake_coords
    st r3, r2 # update address start_snake_coords in cyclic buffer
    pop r0 # return r0 its correct value
    rts

move_up> # move the snake's head up
    ldi r1, 0b11110000
    and r0, r1
    ldi r2, 0b11110000 # when head stands next to board, Y coord is equal
to 0b1111
    if
        cmp r2, r1
    is eq
        jsr game_over # when we can't move and the user loses
    else
        ldi r1, 0x10
        add r1, r0 # otherwise we increment Y coordinate
    fi
    rts

move_bottom> # move the snake's head down, structure is equal to subroutine
above
    ldi r1, 0b11110000
    and r0, r1
    if
```

```
        tst r1
    is eq
        jsr game_over # if Y coord = 0 we can't move down
    else
        ldi r1, 0x10
        sub r0, r1
        move r1, r0
    fi
    rts

move_left> # move the snake's head to the left, structure is equal to
subroutine above
    ldi r1, 0b1111
    and r0, r1
    if
        tst r1
    is eq
        jsr game_over # if X coord = 0 we can't move left
    else
        ldi r1, 0b1
        sub r0, r1
        move r1, r0
    fi
    rts

move_right> # move the snake's head to the right, structure is equal to
subroutine above
    ldi r1, 0b1111
    and r0, r1
    ldi r2, 0b1111
    if
        cmp r2, r1
    is eq
        jsr game_over # if X coord = 0b1111 we can't move left
    else
        ldi r1, 1
        add r1, r0
    fi
    rts

move_snake> # the main function of the snake's movement
```

```
    ldi r1, external_devices_data # addres of keyboard input in memory
    ldb r1, r1
    if
        tst r1
    is z
        if
            tst r4
        is z
            rts # if we have 0 from keyboard and 0 from current direction,
it means that snake hasn't started moving yet
        else
            move r4, r1 # if we get 0 from keyboard and snake has
direction, snake moves in previous direction
        fi
    fi

    move r1, r4 # saving the current direction as the direction of the
snake

        # determining and calculating the new coords of the snake's head,
        # in which direction we need to move the snake
        # (in r2 values from keyboard, check docs for more info)

    ldi r2, 0b1
    if
        cmp r1, r2
    is eq
        jsr move_up
    else
        ldi r2, 0b10
        if
            cmp r1, r2
        is eq
            jsr move_right
        else
            ldi r2, 0b100
            if
                cmp r1, r2
            is eq
                jsr move_bottom
            else
```

```
                jsr move_left
            fi
        fi
    fi

    ldi r1, external_devices_data + 1 # getting current food's coords
    ldb r1, r1
    if
        cmp r1, r0
    is eq # if new head coords is equal to those food, we push new pixel in
snake and eat the food
        jsr push_coords
        jsr eat_food
    else # otherwise we check: did the snake crash into itself?
        if
            jsr check_pixel
        is z # the new snake's head pixel isn't already exists, so let's
simply move the snake
            jsr push_coords # push new pixel in the snake
            jsr pop_coords # pop last pixel from the snake
        else # the new snake's head pixel is already exists, so snake
crashed into itself -> game over
            jsr game_over
        fi
    fi
    rts

init_snake> # snake's initialization
    ldi r0, 0 # init start coords
    jsr push_coords # push its to snake
    ldi r1, 1 # init snake length 1
    ldi r2, snake_length
    st r2, r1
    clr r4 # snake hasn't direction at the beginning
    rts

set_external_devices> # set address for memory mapped input
    ldi r5, external_devices_data
    ldi r6, 0
    ldi r6, 0b1000 # save address in memory mapped module from r5 (see
IO-api docs)
```

```
    ldi r6, 0
    rts

check_pixel> # getting current state of current pixel in r0, store res to
r1
    ldi r6, 0b10 # turn on checking pixels on display controller (see the
docs)
    ldi r1, external_devices_data + 2
    ldb r1, r1 # getting result from display controller
    ldi r6, 0 # turn off checking
    tst r1 # for branches
    rts

generate_food> # genenrating new food, previous food stays at the display,
because now it's part of the snake
    push r0 # save r0 value, because it's current head's coordinates
    do
        ldi r6, 1
        ldi r6, 0 # generating new 8-bit length value from random generator
(see the docs)
        ldi r1, external_devices_data + 1
        ldb r1, r0 # getting result
        jsr check_pixel # if this pixel is already exist, we need to
regenerate it
    until z
    jsr turn_on_pixel
    jsr push_coords_to_display # turn on this pixel on display
    pop r0 # return r0 correct value
    rts

game_over> # game over logic
    ldi r5, 0b100
    or r5, r6 # setting game over screen (docs)
    halt # stopping the processor
    rts

win> # win logic
    halt # stopping the processor
    rts

eat_food>
```

```
    ldi r1, snake_length
    ld r1, r1
    inc r1 # increment the snake's length
    ldi r2, snake_length
    st r2, r1
    ldi r6, 0b10000
    clr r6 # increment the user's scores
    ldi r1, external_devices_data + 3
    ldb r1, r1 # getting value from byte, which contains win event (1 or 0)
    tst r1
    bnz win # checking, did the user win
    jsr generate_food # generating new food
    rts

main>
    jsr set_external_devices
    jsr init_snake
    jsr generate_food # generating first food
    while
        ldi r1, snake_length
        ld r1, r1
        ldi r2, 256 # display size (16 x 16)
        cmp r2, r1
    stays gt # we work until the snake fills the entire display
        jsr move_snake
    wend
end.
```